FUCAPE
BUSINESS SCHOOL

Processamento de Consultas Lógicas, Posicionais e Rankeadas

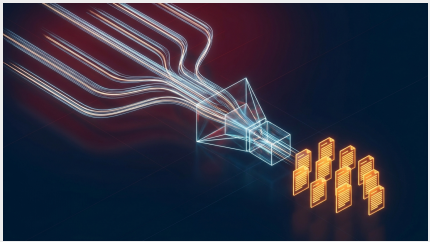
PLN e Sistemas de Recuperação de Informação

Prof. Dr. Wagner Perin
FUCAPE Business School

Processamento de Consultas Lógicas, Posicionais e Rankeadas

Índices Posicionais, Seleção Top-K, Tiered Indexes e Algoritmo WAND

Disciplina: PLN e Sistemas de Recuperação da Informação
Programa: Ciência de Dados para Negócios | FUCAPE
Carga Horária: 4 h/a (200 min) **Prof.:** Prof. Dr. Wagner Perin
Unidade Curricular: UNIDADE 3 — Resolução e Otimização de Consultas



INSIGHT EXECUTIVO

Objetivo Central da Aula:
Dominar a resolução de consultas booleanas complexas, índices posicionais para frases exatas e proximidade, estruturas de ranking Top-K via Min-Heap e algoritmos de poda dinâmica segura (WAND e Block-Max WAND) capazes de eliminar mais de 95% do custo computacional de CPU.

Resgate da Aula Anterior & O Novo Desafio em Tempo de Consulta

Da Compressão em Repouso ao Gargalo de Avaliação em Tempo Real

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Dilema Arquitetural: Na Aula Anterior, reduzimos o tamanho do índice em 75% com d-gaps e Variable Byte Encoding, mantendo as listas residentes na RAM. Mas ter o índice na memória resolve sozinho a precisão de frases e a latência de cauda sob tráfego intenso?

Da Compressão em Repouso ao Gargalo de Avaliação em Tempo Real:

- **Conquista da Aula Anterior:** Vencemos o *Memory Wall* compactando dicionário (*Front Coding*) e postagens (*Variable Byte*), eliminando I/O lento de disco.
- **Gargalo Aberto 1 (Precisão Sintática):** Nosso índice registra apenas **quais documentos** contêm cada termo, perdendo a ordem e contiguidade das palavras no texto.
- **Gargalo Aberto 2 (Exaustão de CPU no Ranking):** Em grandes acervos, uma busca com múltiplos termos mobiliza milhões de documentos candidatos.
- **A Pergunta de Engenharia:** Como encontrar os **Top-K documentos em < 10 ms** sem pontuar e ordenar exaustivamente toda a base candidata?

A Dor de Negócio I — Falsos Positivos Jurídicos

A Falha Crítica do Modelo Bag-of-Words em Buscas por Frases Exatas

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Caso Real 1 (Crise de Precisão em LegalTech): Escritórios de advocacia corporativa ameaçam cancelar contratos devido ao excesso de ruído em buscas por teses jurídicas.



A Falha do Modelo Booleano Não-Posicional:

- **A Consulta do Advogado:** Busca pela tese exata `"dano moral coletivo"`.
- **O Comportamento do Motor (`dano AND moral AND coletivo`):** Retorna acórdão de 80 páginas porque:
  - *"dano"* aparece na **página 2** (*"dano material ao erário"*);
  - *"moral"* aparece na **página 35** (*"dever moral da administração"*);
  - *"coletivo"* aparece na **página 78** (*"transporte coletivo"*).
- **Impacto no Negócio:** Embora a interseção booleana seja verdadeira no nível do documento, o resultado é um **falso positivo semântico** prejudicial.

A Dor de Negócio II — Exaustão de CPU no Top-K

Colapso de Latência e Desperdício no Ranking Exaustivo em E-commerce

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Caso Real 2 (Black Friday em Marketplace): Sob 4.000 buscas/segundo em catálogo de 50 milhões de SKUs, os servidores atingem 100% de CPU.



A Anatomia da Crise de Processamento:

- **Consulta Popular:** `"smartphone 5g tela amoled"`.
- **Volume de Candidatos:** Mais de **600.000 produtos** possuem pelo menos uma dessas palavras.
- **Desperdício Computacional:**
  - O usuário enxerga apenas os **Top-20 produtos** ($K = 20$).
  - O motor calcula o score BM25 completo para todos os **600.000 candidatos** e ordena um vetor de 600.000 posições!
- **Resultado: 99,996% dos cálculos vão para o lixo**; a latência $p99$ sobe de 18 ms para 1.400 ms.

A Matemática do Desperdício: Ordenação vs. Min-Heap

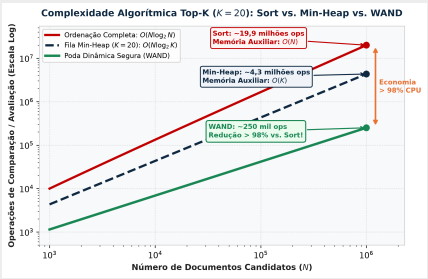
Análise de Complexidade Assintótica para Seleção Top-K ($K \ll N$)

Comparativo Assintótico ($N = 10^6$ candidatos, $K = 20$):

Estratégia	Tempo	Operações	Memória
1. Sort Completo	$O(N \log N)$	$\approx$ **20 M**	$O(N)$
2. Min-Heap Top-K	$O(N \log K)$	$\approx \mathbf{4,3}$ **M**	$\mathbf{O(K)}$ (20 itens)
3. Poda WAND	**Sublinear**	$\approx \mathbf{250}$ **k**	$\mathbf{O(K)}$ (20 itens)

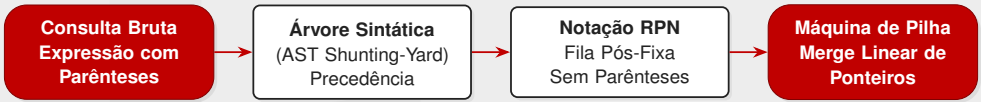
INSIGHT EXECUTIVO

Intuição da Corrida Classificatória:
Se a nota de corte para entrar no Top-10 é 10,0s e um corredor já gastou 9,8s na metade da pista, não precisamos esperar ele cruzar a linha de chegada para saber que está eliminado. Essa é a essência da poda dinâmica!



Avaliação de Consultas Booleanas Complexas

Árvores Sintáticas (AST) e Avaliação Linear via Pilha (RPN)



A Mecânica da Pilha Pós-Fixa (Reverse Polish Notation - RPN):

- **Eliminação de Parênteses:** O algoritmo *Shunting-Yard* converte a consulta em notação pós-fixa sem ambiguidades de precedência.
- **Execução em Pilha:** Ao ler um termo, empilha seu ponteiro de postagens; ao ler um operador (`AND`, `OR`, `NOT`), desempilha os dois operandos do topo, executa o merge linear e empilha a lista resultante.
- **Otimização do Planejador (*Query Planner*):** Em blocos conjuntivos (`A AND B AND C`), reordena os termos pela **menor frequência documental (**DF_t**)** primeiro para minimizar o tamanho dos resultados intermediários!

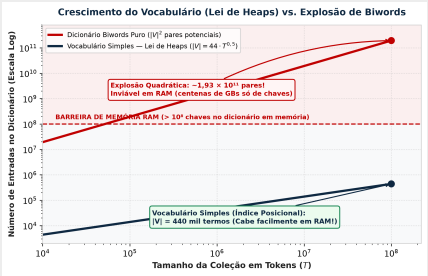
Por que Índices de Bigramas Falham em Escala?

A Explosão Combinatória Quadrática $|V|^2$ no Dicionário

Tentativa Ingênua: Indexar pares de palavras consecutivas no vocabulário (`"dano moral"`, `"moral coletivo"`).

Os 3 Colapsos de Engenharia dos *Biwords*:

1. **Explosão Quadrática na Lei de Heaps ($|V|^2$):**
   Para vocabulário de $|V| = 440.000$ palavras, o espaço potencial de pares atinge $|V|^2 \approx \mathbf{1,93 \times 10^{11}}$ termos, inviabilizando a RAM do léxico!
2. **Falsos Positivos em Frases Longas:**
   `"dano moral coletivo"` decomposto em `"dano moral" AND "moral coletivo"` aceita textos onde os pares ocorrem em páginas distantes.
3. **Rigidez Sintática:**
   Incapaz de resolver buscas por proximidade flexível (k tokens de separação).

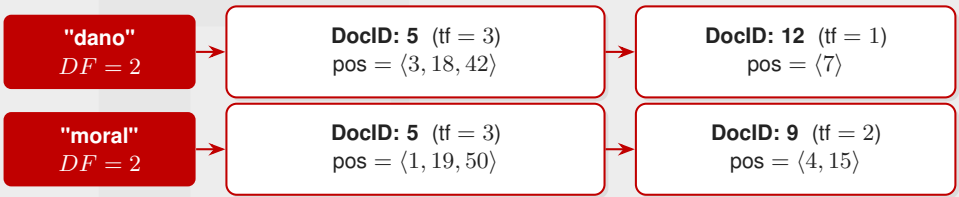


Anatomia do Índice Invertido Posicional

Coordenadas de Tokens nas Postagens e Trade-off de Armazenamento

Estrutura Hierárquica de uma Postagem Posicional:

$$\text{Termo } t \longrightarrow \Big\langle \text{DocID},\ tf_{t,d},\ [\text{pos}_1, \text{pos}_2, \ldots, \text{pos}_{tf}] \Big\rangle$$



- **Trade-off Físico:** Armazena coordenadas para cada ocorrência de token (T), aumentando o índice em $2\times$ a $4\times$.
- **Mitigação via Aula Anterior:** Como as posições são estritamente crescentes, codificam-se via **d-gaps posicionais** com **Variable Byte Encoding**, mantendo alta densidade!

Consultas por Frase Exata (Phrase Queries)

Condição Matemática Estrita de Contiguidade e Direcionalidade

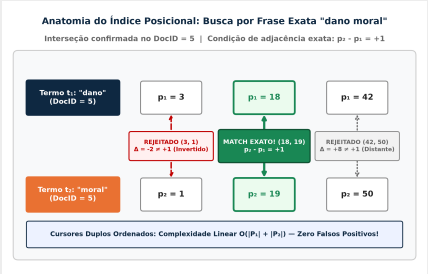
Condição Estrita para Dois Termos (`"t1 t2"`):

Dado um documento d comum às listas, existe casamento de frase exata se existirem posições $p_1 \in \text{pos}(t_1, d)$ e $p_2 \in \text{pos}(t_2, d)$ tais que:

$$\mathbf{p_2 - p_1 = 1 \iff p_2 = p_1 + 1}$$

Generalização para m Termos (`"t1..tm"`):

$$\mathbf{p_i = p_{i-1} + 1, \quad \forall i \in \{2, \ldots, m\}}$$



INSIGHT EXECUTIVO

Propriedade da Assimetria:
A diferença deve ser $+1$. Se $p_2 - p_1 = -1$, os termos estão em ordem invertida (*"moral dano"*), sendo descartados com rigor matemático!

Consultas por Proximidade (Proximity Search)

Operadores de Janela Deslizante e o Implicit Proximity Boost no Ranking

Formalização de Janelas de Tolerância Posicional (k tokens):

- **1. Proximidade Simétrica (Não-Direcional — `NEAR/k` ou `~k`):** $|\mathbf{p_1} - \mathbf{p_2}| \leq \mathbf{k}$.
  Exemplo: `"dividendos proventos"~5` recupera *"pagamento de dividendos e outros proventos"* ($|p_1 - p_2| = 4 \leq 5$).
- **2. Proximidade Direcional Ordenada:** $\mathbf{1} \leq \mathbf{p_2} - \mathbf{p_1} \leq \mathbf{k}$. Exige que t_1 preceda t_2 dentro de k tokens.

INSIGHT EXECUTIVO

Insight de Ranking Executivo (Implicit Proximity Boost):

Motores industriais (Elasticsearch `match_phrase` com `slop`, Google) utilizam o inverso da distância posicional $\left(\frac{1}{|p_1 - p_2|}\right)$ como um multiplicador de relevância BM25: documentos com termos adjacentes recebem bônus imediato de pontuação!

Trace Visual da Interseção Posicional

Varredura em Tempo Linear $O(|P_1| + |P_2|)$ sem Combinações Quadráticas

Frase Exata `"dano moral"` **(**$k = 1$**, direcional) no** `DocID = 5`**:**

$P_1 = [\mathbf{3}, \mathbf{18}, \mathbf{42}]$ (cursor p_i) | $P_2 = [\mathbf{1}, \mathbf{19}, \mathbf{50}]$ (cursor p_j)

Iteração	Posição $p_1 = P_1[p_i]$	Posição $p_2 = P_2[p_j]$	Diferença ($\Delta = p_2 - p_1$)	Teste ($\Delta == 1$?)	Decisão de Avanço
1	$p_1 = 3$	$p_2 = 1$	$1 - 3 = -2$	Falso ($p_2 < p_1$)	Avança $p_j \rightarrow$ aponta para 19
2	$p_1 = 3$	$p_2 = 19$	$19 - 3 = +16$	Falso ($p_1 < p_2$)	Avança $p_i \rightarrow$ aponta para 18
3	$\mathbf{p_1 = 18}$	$\mathbf{p_2 = 19}$	$\mathbf{19 - 18 = +1}$	**MATCH EXATO!**	Registra par; avança p_i e p_j
4	$p_1 = 42$	$p_2 = 50$	$50 - 42 = +8$	Falso ($p_1 < p_2$)	Avança $p_i \rightarrow$ Fim da lista!

INSIGHT EXECUTIVO

Eficiência Garantida: Evita a comparação exaustiva de todos os pares ($O(|P_1| \times |P_2|)$), percorrendo as listas em tempo linear estrito $O(|P_1| + |P_2|)$.

Reflexão Analítica & Decisão de Arquitetura 1

Arbitrando Trade-offs de Indexação Posicional em Sistemas Críticos

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO – Dilemas de Engenharia em Dados Corporativos

1. **O Dilema das *Stop Words* em Frases Exatas:**
   Se a equipe de dados remover palavras funcionais (*"sem"*, *"de"*, *"com"*) para economizar memória, como distinguir `"rescisão sem justa causa"` de `"rescisão com justa causa"`?
   $\Longrightarrow$ **Decisão de Engenharia:** Em índices posicionais corporativos, **preservamos stop words** ou mantemos seus saltos de offset (`position_increment_gap`), garantindo separação semântica correta!

2. **Por que a Interseção Posicional nunca deve anteceder a de DocIDs?**
   As listas de posições só são descomprimidas e percorridas **após** a confirmação de que $d_1 == d_2$, poupando mais de 95% das leituras de offsets!

Estratégias de Varredura de Índices — TAAT vs. DAAT

Term-at-a-Time vs. Document-at-a-Time na Resolução de Consultas Rankeadas

Term-at-a-Time (TAAT): Varredura Horizontal

- Processa 1 lista invertida inteira por vez.
- Acumula scores em tabela hash global.
- **Memória Auxiliar:** $O(N_{\text{candidatos}})$ acumuladores em RAM.
- **Poda Dinâmica:** Difícil (scores incompletos até o fim).

Document-at-a-Time (DAAT): Varredura Vertical

- Avança cursores em paralelo em todas as listas.
- Calcula score final completo de 1 DocID por vez.
- **Memória Auxiliar: Mínimo** $O(K)$ (apenas Min-Heap Top-K).
- **Poda Dinâmica: Nativa (Base do algoritmo WAND e Lucene)**.

INSIGHT EXECUTIVO

Conclusão de Engenharia:
O modelo **DAAT** é o padrão dos motores modernos porque viabiliza a avaliação seletiva com saltos dinâmicos e consumo de memória rigorosamente delimitado a $O(K)$.

Seleção Top-K Dinâmica com Min-Heap e Limiar θ

Manutenção da Fila de Prioridade em Tempo Logarítmico $O(\log_2 K)$

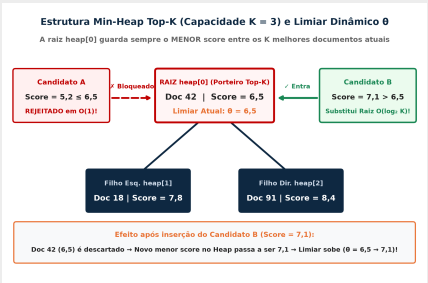
A Mecânica da Fila de Prioridade (*Min-Heap*):

- Capacidade fixa delimitada a K **elementos**.
- **A Raiz (`heap[0]`):** Guarda sempre o menor score entre os K melhores atuais.
- **O Limiar Dinâmico (θ):**

$$\theta = \begin{cases} 0,0, & \text{se } |\text{Heap}| < K \\ \text{Heap}[0], & \text{se } |\text{Heap}| = K \end{cases}$$

Regra de Atualização em $O(\log_2 K)$:

- Se Score$(d) \leq \theta$: **Descarte imediato em $O(1)$!**
- Se Score$(d) > \theta$: Executa `heappushpop` em $O(\log_2 K)$, elevando o novo θ!



Poda Estática e Heurística I (Inexact Top-K)

Champion Lists e Postagens Ordenadas por Impacto

Estratégias Pré-Computadas em Tempo de Indexação:

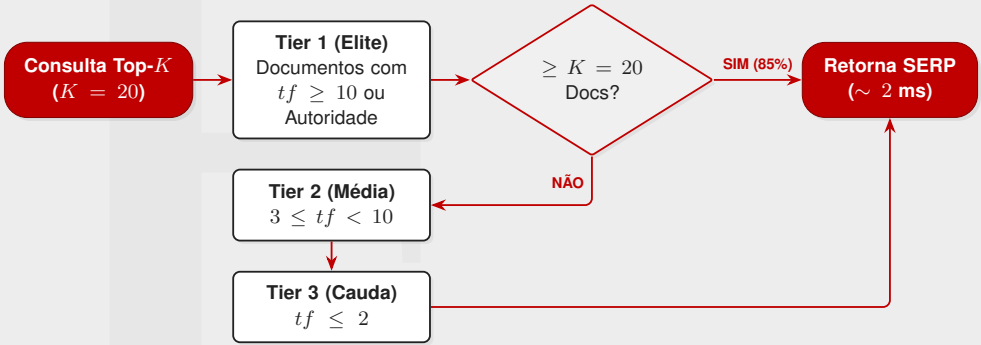
- **1. Champion Lists (Listas de Campeões / *Fancy Lists*):**
  - Para cada termo, pré-calculamos os r documentos de maior BM25 (ex: $r = 500$) em lista separada.
  - *Risco (Inexact Top-K):* Um documento que seja o 501° em dois termos isolados, mas o 1° na soma combinada, pode ser ignorado!
- **2. Postagens Ordenadas por Impacto (*Impact-Ordered Postings*):**
  - Postagens ordenadas decrescentemente pelo score BM25 em vez de DocID crescente.
  - Viabiliza parada antecipada (*Early Termination*), mas torna o merge de múltiplos termos mais complexo.

INSIGHT EXECUTIVO

O Trade-off das Heurísticas:
São extremamente rápidas, mas não oferecem garantias matemáticas de exatidão do ranking final.

Poda Estática e Heurística II — Índices em Camadas

Organização Hierárquica por Relevância e Parada Antecipada



Vantagens dos Índices em Camadas (*Tiered Indexes*):

- **Mais de 85% das consultas comerciais** são resolvidas na Camada Tier 1 ($< 5\%$ do tamanho total do índice!).
- Evita que documentos marginais (com 1 menção no rodapé) onerem consultas de alto volume.

Poda Dinâmica Segura — Fundamentos do WAND

Weak AND (Broder et al., 2003): Poda Agressiva com 100% de Exatidão

A Invenção do Algoritmo WAND (*Weak AND*):

Diferente das heurísticas inexatas, o WAND é um algoritmo de **Poda Dinâmica Segura (*Rank-Safe Pruning*)**: devolve exatamente o mesmo Top-K da busca exaustiva.

1. O Teto Matemático por Termo (*Upper Bound* UB_t) e a Desigualdade de Poda:

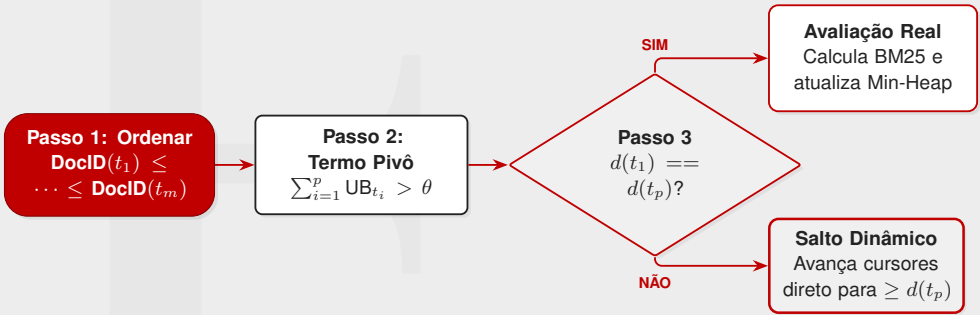
$$\mathrm{UB}_{\mathbf{t}} = \max_{\mathbf{d} \in \mathbf{L_t}} \mathrm{Score}_{\mathbf{BM25}}(\mathbf{t}, \mathbf{d}) \quad \Longrightarrow \quad \mathrm{Score}(\mathbf{d}, \mathbf{q}) \leq \sum_{\mathbf{t} \in \mathbf{q} \cap \mathbf{d}} \mathrm{UB}_{\mathbf{t}}$$

INSIGHT EXECUTIVO

Consequência Fundamental:
Se a soma dos tetos de um conjunto de termos for menor ou igual a θ, o documento é **matematicamente incapaz** de entrar no Top-K e pode ser descartado sem pontuação!

A Mecânica do Algoritmo WAND — Os 3 Passos

Loop Central: Ordenação de Cursores, Descoberta do Pivô e Salto Dinâmico



Detalhamento dos 3 Passos Operacionais:

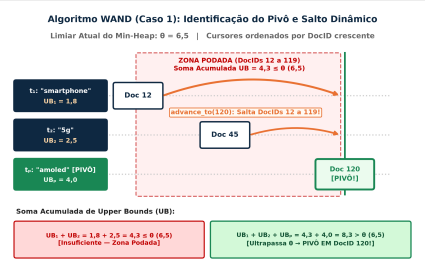
1. **Ordenação de Cursores:** Termos ativos ordenados pelo `DocID` apontado no momento.
2. **Descoberta do Pivô (t_p):** Primeiro índice p onde o acumulado dos tetos supera θ.
3. **Decisão de Salto ou Avaliação:** Se t_1 e t_p apontam para o mesmo doc, avalia; se desalinhados, salta!

WAND Passo a Passo — Caso 1: Salto Dinâmico

Desalinhamento de Cursores e Prova Matemática do Salto

Cenário de Desalinhamento: DocID$(t_1) <$ DocID(t_p)

- t_1 (`"smartphone"`): Cursor em DocID 12 ($\text{UB}_1 = 1,8$)
- t_2 (`"5g"`): Cursor em DocID 45 ($\text{UB}_2 = 2,5$) $\rightarrow$ Acumulado: $4,3 \leq \theta(6,5)$
- t_p (`"amoled"`): Cursor em DocID 120 ($\text{UB}_3 = 4,0$) $\rightarrow$ Acumulado: $8,3 > \theta(6,5)$ **[PIVÔ]**



A Prova Matemática do Salto:

- Todo documento em $[12, 119]$ pode conter no máximo t_1 e t_2.
- Nota máxima teórica: $1,8 + 2,5 = 4,3 \leq \theta\ (6,5)$.
- **Ação do WAND:** Salta cursores de t_1 e t_2 diretamente para ≥ 120, **pulando 108 documentos** com zero perda de relevância!

WAND Passo a Passo — Caso 2: Alinhamento

Convergência de Cursores, Avaliação Real e o Efeito Bola de Neve

Cenário de Convergência: DocID$(t_1) ==$ DocID$(t_p) = d_{\text{pivô}} = 120$

Após o salto, os cursores se alinham no mesmo documento candidato com potencial matemático.

Procedimento de Avaliação e Elevação do Limiar:

1. **Cálculo do Score BM25 Real:** Como $d_{\text{pivô}} = 120$ tem potencial $(8, 3 > 6, 5)$, somam-se os scores reais dos termos presentes e avançam-se os cursores.
2. **Teste de Admissão no Min-Heap:** Se o score real (ex: $7, 1$) superar $\theta = 6, 5$, substitui o pior elemento no topo do heap.
3. **O Efeito Bola de Neve (*Snowball Effect*):** O novo limiar θ sobe para $6, 9$. Quanto maior θ, mais para a direita o pivô se desloca nas próximas iterações, gerando **saltos cada vez maiores e mais frequentes**!

INSIGHT EXECUTIVO

Garantia Teórica:
Nenhum documento do Top-K verdadeiro é descartado, pois apenas documentos com teto teórico comprovadamente inferior a θ são saltados.

Reflexão Analítica & Decisão de Arquitetura 2

Exercício Prático Guiado de Cálculo do Pivô WAND em Sala de Aula

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO – Exercício Guiado: Estado Atual de Busca Top-10 com Limiar $\theta = 9,0$

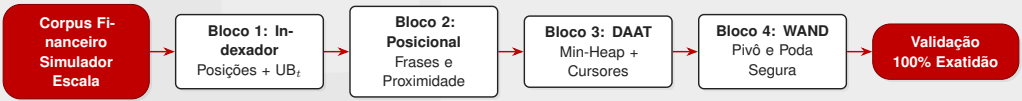
Cursor (i)	Termo (t_i)	DocID Atual	Teto (UB_{t_i})	Soma Acumulada	Supera $\theta = 9,0$?
1	t_1: `"credito"`	DocID 15	$2,0$	$2,0$	Não ($2,0 \leq 9,0$)
2	t_2: `"inadimplencia"`	DocID 40	$3,5$	$5,5$	Não ($5,5 \leq 9,0$)
3	t_3: `"provisao"`	DocID 100	$5,0$	$\mathbf{10,5}$	**SIM!** ($10,5 > 9,0$)

Resolução Analítica Guiada:

1. **Quem é o Termo Pivô?** É t_3 (`"provisao"`), apontando para $d_{\text{pivô}} = \mathbf{100}$.
2. **Qual é a decisão do WAND?** Como DocID$(t_1) = 15 < 100$, avançamos os cursores de t_1 e t_2 para ≥ 100, **podando sem ler todos os documentos no intervalo** $[15, 99]$.
3. **E se a soma total de todos os termos ativos for** $\leq 9,0$**?** Nenhum pivô é encontrado e o WAND encerra a busca imediatamente (**Early Termination Total**)!

Arquitetura da Solução Prática em Python

Engenharia do Motor Posicional e WAND Top-K em 4 Blocos Modulares



Roteiro das 4 Micro-Etapas Práticas:

- **Bloco 1 (Slide 23):** Indexação posicional e pré-cálculo dos tetos UB_t do Okapi BM25.
- **Bloco 2 (Slide 24):** Algoritmo de interseção posicional linear com ponteiros duplos.
- **Bloco 3 (Slide 25):** Gestão do Min-Heap (`heapq`) e funções de cursor (`current_doc` e `advance_to`).
- **Bloco 4 (Slide 26):** Núcleo do motor WAND com descoberta de pivô e telemetria de saltos.

Laboratório — Bloco 1: Indexação Posicional e Upper Bounds

Micro-Etapa 1: Posições de Tokens e Pré-cálculo do Teto UB_t do BM25

```
class PositionalAndWANDIndex:
    def finalize_and_compute_bounds(self):
        """Ordena postagens por DocID e pre-calcula scores BM25 e Upper Bounds (UB_t)."""
        self.avg_dl = sum(self.doc_lengths.values()) / max(1, self.N)
        for term, plist in self.postings.items():
            plist.sort(key=lambda x: x['doc_id'])
            df_t = len(plist)
            idf = math.log(1.0 + (self.N - df_t + 0.5) / (df_t + 0.5))
            max_score = 0.0
            for entry in plist:
                dl, tf = self.doc_lengths[entry['doc_id']], entry['tf']
                tf_norm = (tf * (self.k1 + 1.0)) / (tf + self.k1 * (1.0 - self.b + self.b * (dl / self.avg_dl)))
                entry['score'] = idf * tf_norm
                if entry['score'] > max_score:
                    max_score = entry['score']
            self.upper_bounds[term] = max_score # Teto matematico UB_t do termo!
```

Destaques de Implementação:

- Cada entrada em `self.postings[term]` armazena `{'doc_id', 'tf', 'positions', 'score'}`.
- `self.upper_bounds[term]` guarda o $\max_d \text{Score}_{BM25}(t, d)$ para alimentar a escolha do pivô no WAND.

Laboratório — Bloco 2: Algoritmo de Interseção Posicional

Micro-Etapa 2: Varredura Sincronizada de Ponteiros Externos e Internos

```python
def positional_intersect(self, term1: str, term2: str, k: int = 1, directional: bool = True):
    p1, p2 = self.postings.get(term1, list()), self.postings.get(term2, list())
    i, j, matches = 0, 0, list()
    while i < len(p1) and j < len(p2):
        d1, d2 = p1[i]['doc_id'], p2[j]['doc_id']
        if d1 == d2: # Mesmo documento! Inspeciona posicoes internas:
            pos1, pos2, pi, pj = p1[i]['positions'], p2[j]['positions'], 0, 0
            while pi < len(pos1) and pj < len(pos2):
                diff = pos2[pj] - pos1[pi]
                cond = (1 <= diff <= k) if directional else (abs(diff) <= k)
                if cond:
                    matches.append((d1, pos1[pi], pos2[pj]))
                    pi += 1; pj += 1
                elif pos1[pi] < pos2[pj]: pi += 1
                else: pj += 1
            i += 1; j += 1
        elif d1 < d2: i += 1
        else: j += 1
    return matches
```

Destaques de Implementação:

- **Frase Exata:** Invocação com `k=1` e `directional=True` ($\mathbf{p_2 - p_1 == 1}$).
- **Proximidade Simétrica:** Invocação com `k=5` e `directional=False` ($|\mathbf{p_2 - p_1}| \leq \mathbf{5}$).

Laboratório — Bloco 3: Min-Heap Top-K e Cursores do WAND

Micro-Etapa 3: Fila de Prioridade `heapq` e Navegação de Cursores DAAT

```
cursors = {t: 0 for t in q_terms}
min_heap = list() # Guarda tuplas (score, doc_id) dos K melhores atuais
threshold = 0.0  # Limiar dinamico Teta (raiz do Min-Heap quando cheio)

def current_doc(term: str) -> float:
    idx, plist = cursors[term], self.postings[term]
    return plist[idx]['doc_id'] if idx < len(plist) else float('inf')

def advance_to(term: str, target_doc: int):
    """Salta o cursor da lista diretamente para o primeiro DocID >= target_doc."""
    plist, idx = self.postings[term], cursors[term]
    while idx < len(plist) and plist[idx]['doc_id'] < target_doc:
        idx += 1
    cursors[term] = idx
```

Por que a Estrutura Min-Heap é Ideal para Top-K?

- `min_heap[0][0]` acessa o limiar θ em tempo constante $\mathbf{O(1)}$.
- `heapq.heappushpop` substitui o pior elemento em tempo logarítmico $\mathbf{O(\log_2 K)}$.

Laboratório — Bloco 4: O Motor de Poda Dinâmica WAND

Micro-Etapa 4: Loop Central, Descoberta do Pivô e Salto Dinâmico

```
while True:
    active = [t for t in q_terms if current_doc(t) != float('inf')]
    if not active: break
    active.sort(key=lambda t: current_doc(t)) # Passo 1: Ordena por DocID atual

    # Passo 2: Encontra o Termo Pivo onde soma(UB_t) > threshold
    cum_ub, pivot_idx = 0.0, None
    for i, t in enumerate(active):
        cum_ub += self.upper_bounds[t]
        if cum_ub > threshold:
            pivot_idx = i; break
    if pivot_idx is None: break # Parada antecipada total!

    pivot_doc = current_doc(active[pivot_idx])
    if current_doc(active[0]) == pivot_doc: # Passo 3A: Alinhados -> Avalia!
        doc_score = sum(self.postings[t][cursors[t]]['score'] for t in active if current_doc(t) == pivot_doc)
        for t in active:
            if current_doc(t) == pivot_doc: cursors[t] += 1
        # Atualiza Min-Heap e eleva threshold...
    else:                                   # Passo 3B: Desalinhado -> Salto!
        for i in range(pivot_idx): advance_to(active[i], pivot_doc)
```

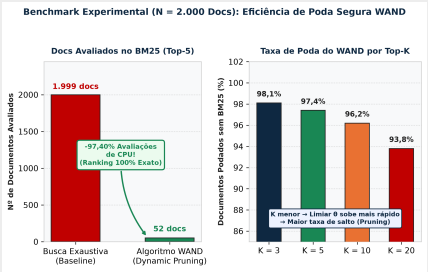
Validação Experimental & Benchmark de Escala

Comparativo de Desempenho: Busca Exaustiva vs. WAND (2.000 Docs)

Saída Real do Benchmark no Corpus:

Benchmark de Escala Top-5 (N = 2.000 Docs)

```
Consulta:  'mercado financeiro crescimento debentures superavit'
------------------------------
Top-5 Identico?  True (100% concordancia)
Candidatos Exaustivos:  1.999 documentos pontuados
Avaliados pelo WAND: 52 documentos pontuados
Saltos de Poda (Skips):  98 saltos dinamicos executados
------------------------------
Economia Real de CPU: 97,40% de calculos BM25 evitados!
```

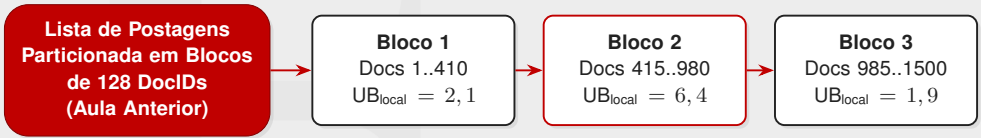


INSIGHT EXECUTIVO

Conclusão Experimental:
O WAND retornou **exatamente os mesmos 5 documentos** e pontuações com precisão de 4 casas decimais, poupando **97,40% das operações de pontuação**!

Estado da Arte Industrial — Block-Max WAND

Como o Apache Lucene e Elasticsearch Unem Blocos de 128 com o WAND



Por que o Block-Max WAND (BMW - Ding & Suel, 2011) Revolucionou a Indústria?

- **Imunidade a Outliers:** Se um único documento tiver score extremo, o WAND clássico infla o teto global UB_t. O BMW armazena o teto local UB_{bloco} no cabeçalho do bloco de 128 itens.
- **Salto sem Descompressão de Bytes:** Se $UB_{bloco} \leq \theta$, o motor **nem sequer descomprime os bytes do bloco**, economizando barramento de memória e ciclos de descompressão!

FinOps Executivo — Multiplicação de QPS e Custos AWS

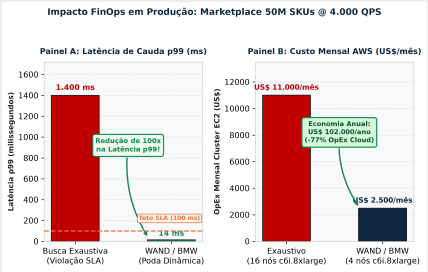
Impacto Financeiro da Poda Dinâmica em Alta Concorrência (4.000 QPS)

Impacto Financeiro no Marketplace (50M SKUs):

Dimensão de FinOps	Sem WAND	Com WAND	Ganho / ROI
Docs Pontuados / Busca	600.000	≈ 18.000	**-97% de CPU**
Latência de Cauda ($p99$)	1.400 ms	14 ms	**100x mais rápida**
Vazão por Nó (QPS)	250 qps	1.250 qps	**5x capacidade**
Cluster AWS (`c6i.8xlarge`)	16 nós	4 nós	**-US$ 8.500/mês**
Economia Anual (OpEx)	Base	**-77% Custo**	**US$ 102.000/ano**

INSIGHT EXECUTIVO

Posicionamento Executivo C-Level:
A poda dinâmica segura evitou a duplicação emergencial de servidores na Black Friday, garantindo estabilidade de 14ms e gerando **US$ 102 mil/ano de economia líquida** em infraestrutura.



Síntese da Aula 06 & Teaser para a Aula 07

Da Poda Dinâmica de Consultas à Busca Tolerante a Erros

O Que Dominamos na Aula 06:

1. Consultas Booleanas com Árvores Sintáticas (*AST*) e Pilhas *RPN*.
2. **Índices Posicionais** para Frases Exatas ($p_2 - p_1 = 1$) e Proximidade.
3. Varredura **TAAT vs. DAAT**, seleção Top-K via **Min-Heap** e *Tiered Indexes*.
4. Poda Dinâmica Segura com **WAND** e **Block-Max WAND** ($> 95\%$ CPU poupada).



INSIGHT EXECUTIVO

O Gancho Provocativo para a Aula 07:
Nosso motor é ultrarrápido, mas busque por *'dividendoz'* com 'z' ou *'samsumg'* com 'm': o sistema retorna zero resultados! No mundo real, 20% das buscas contêm erros tipográficos. Como encontrar o documento certo em milissegundos mesmo com erros?
$\Longrightarrow$ **Aula 07: Busca Exata e Aproximada (Tolerant Retrieval): Levenshtein, k-gramas, Soundex e Tries!**